

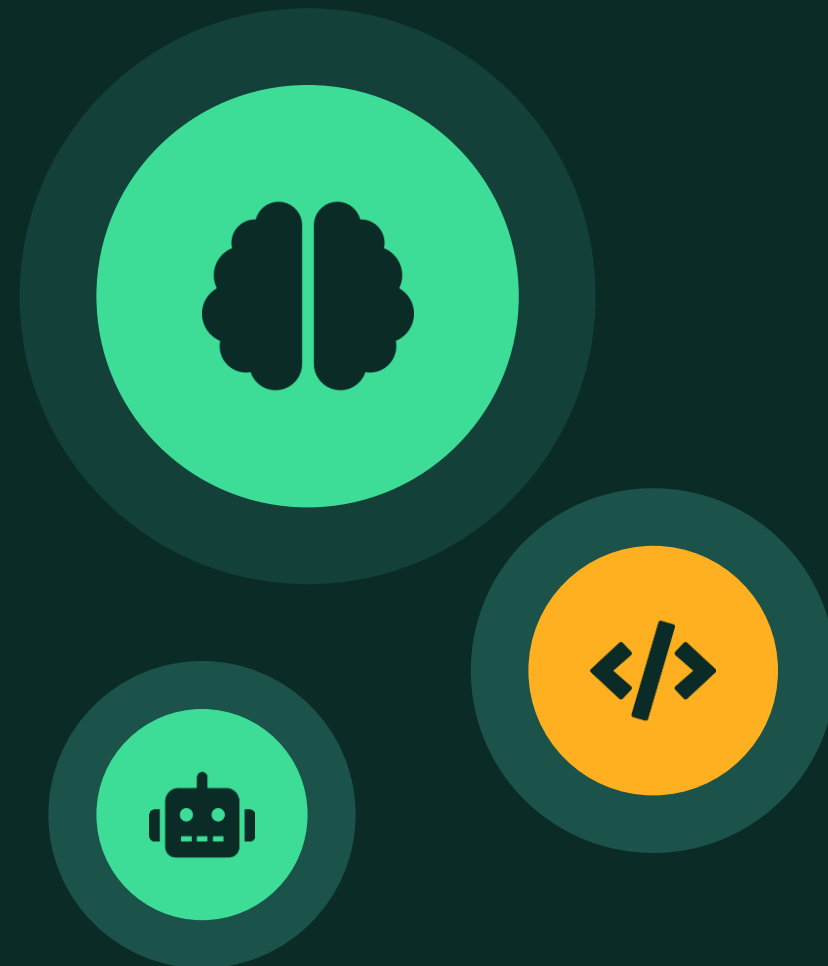
FORMATION INTRA-ENTREPRISE · CGG SERVICES SAS

IA pour les devs

Maîtrisez l'IA pour coder

Jour 2 sur 2 · Industrialiser l'usage de l'IA

Formateur : Daouda DIOP



JOUR 2

Industrialiser l'usage de l'IA dans son workflow de développement



Échauffement : qu'avez-vous essayé depuis hier ? Qu'est-ce qui a fonctionné, qu'est-ce qui vous a surpris ?

5 · Des agents pour des tâches complexes

6 · L'IA dans le workflow d'équipe

7 · Automatiser le cycle de développement

8 · Choisir sa stack IA

Utiliser des agents IA pour des tâches complexes

Travail multi-fichiers, refactoring à grande échelle, tests avancés, automatisation, orchestration multi-agents, et gestion des limites.



Travail multi-fichiers avec les agents

1 Interfaces et types

2 Implémentations

3 Appelants

4 Tests et documentation



Donnez une carte

Quels modules sont concernés.



Énoncez le périmètre et ce qui est hors périmètre



Demandez d'abord un plan transversal aux fichiers



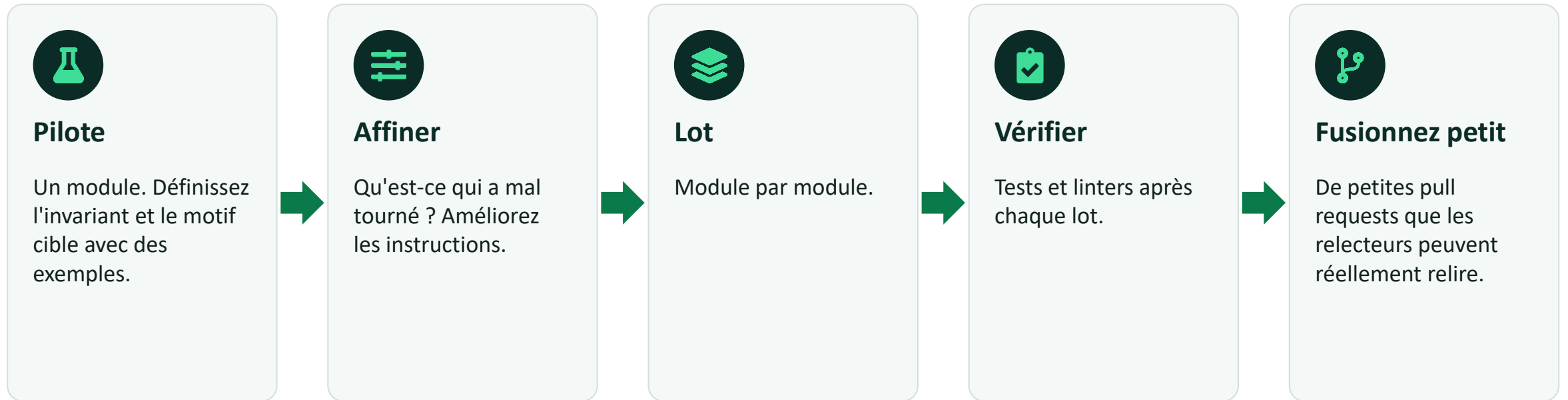
Commitez à chaque étape



Mettez à jour les tests et la documentation dans le même changement

Un bon ordre pour un changement s'étendant sur plusieurs fichiers

Refactoring à grande échelle



Changement mécanique ? Demandez à l'agent d'écrire un script de transformation, puis relisez le script plutôt que des centaines de fichiers édités.

Génération de tests avancés



Trous de couverture

Donnez un rapport de couverture ; demandez des tests sur les branches non testées. Vérifiez qu'ils sont pertinents.



Cas limites et erreurs

Demandez des entrées inhabituelles et des modes d'échec, puis testez-les.



Tests basés sur des propriétés

Là où votre écosystème les supporte : décrivez une propriété qui doit toujours être vraie.



Solidité des tests

Mutation testing : introduisez de petits bugs et vérifiez que les tests échouent.



La couverture mesure quelles lignes ont été exécutées, pas si le comportement a été vérifié.

Automatiser des tâches de développement



Répétable

Elle revient souvent.



Vérifiable

Des tests, des linters ou des vérifications de types peuvent vérifier le résultat.



Faible rayon d'impact

Si ça tourne mal, les dégâts sont limités et réversibles.

Souvent adapté

- Migrations de code
- Boilerplate, scaffolding
- Corrections de lint
- Changelogs
- Mises à jour de documentation

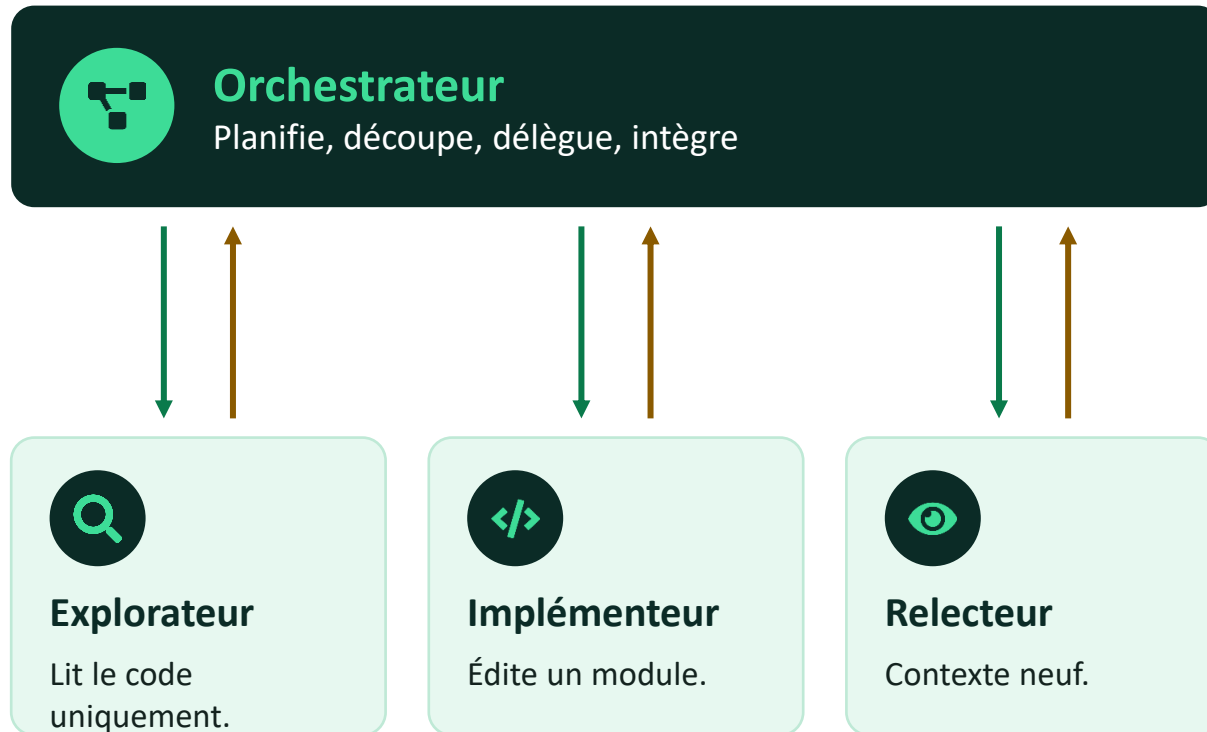
Peu adapté

- Tâches ambiguës
- Aucun moyen de vérifier
- Données de production ou secrets



Faites-le une fois à la main avec l'agent, affinez les instructions, et seulement ensuite automatisez.

Orchestration multi-agents : plusieurs agents, un objectif



Vers le bas : une tâche cadrée Vers le haut : un résultat court. Chaque sous-agent a typiquement son propre contexte.

Quatre motifs courants

- 1 **Orchestrateur + travailleurs**
Un agent découpe la tâche et délègue.
- 2 **Travailleurs parallèles**
Des sous-tâches indépendantes s'exécutent en même temps.
- 3 **Pipeline**
Planificateur, puis implémenteur, puis relecteur.
- 4 **Auteur + relecteur indépendant**
Un second agent vérifie le travail du premier.

 Le multi-agent n'est pas une technologie différente : ce sont plusieurs boucles d'agent, coordonnées.

Quand le multi-agent vaut-il le coup, et quand pas ?



Envisagez-le quand...



Surcharge de contexte

Des sous-agents explorent et retournent des résumés courts.



Sous-tâches indépendantes

Des modules qui changent en parallèle, sans fichiers partagés.



Rôles ou permissions différents

Explorateur en lecture seule, rédacteur, relecteur limité.



Vérification indépendante

Un contexte neuf relit le travail.



Restez avec un seul agent quand...



La tâche est petite ou bien cadrée

La coordination coûte plus qu'elle ne fait gagner.



Les étapes sont couplées ou séquentielles

Chaque étape dépend de la précédente.



Vous ne pouvez pas encore vérifier le résultat

Plus d'agents veut dire plus de sorties à vérifier.



Le coût, la latence, la traçabilité comptent

Plus de tokens, plus d'attente, plus difficile à déboguer.



Commencez avec un seul agent. N'en ajoutez que lorsque vous atteignez une limite concrète.

Faire tourner des dispositifs multi-agents en toute sécurité



Un coût et une latence plus élevés

Plusieurs agents : plus de tokens, plus d'attente.



Les passations perdent de l'information

Un sous-agent ne sait que ce qu'on lui a dit. Rédigez un brief précis.



Des éditions en conflit

Donnez aux agents parallèles des branches ou des copies de travail séparées.



Les erreurs s'accumulent

Une hypothèse fausse faite tôt peut se propager à plusieurs sous-agents.



Plus difficile à suivre

Gardez les tâches et les résultats explicites. Gardez la trace de qui a fait quoi.



Vous relisez toujours tout

Lisez chaque diff et lancez les tests, quel que soit qui a écrit le code.



Une manière pratique de commencer : un orchestrateur, deux ou trois sous-agents étroitement cadrés, des briefs clairs, et les tests comme source de vérité partagée.

Gérer les limites des agents

Problème	Contre-mesure
Dérive de l'objectif	Un périmètre plus resserré, un plan d'abord, l'arrêter et le rediriger
Boucle sur le même échec	Interrompez-le, reformulez avec ce que vous avez appris, ou démarrez une nouvelle session
Contexte surchargé	Des tâches plus petites, des résumés, un redémarrage avec des notes
Prend des actions risquées	Demandes d'approbation pour les commandes sensibles, bac à sable, travail sur une branche
Modifie les tests pour les faire passer	Protégez les tests dans les instructions, lisez toujours le diff en entier
Suit des instructions cachées dans le contenu qu'il lit (injection de prompt)	Traitez le contenu externe comme non fiable ; permissions minimales
Coût et temps	Périmètre restreint ; modèles plus légers pour les tâches simples, quand c'est possible

Six habitudes de sécurité quand un agent a des mains



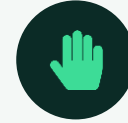
Branche dédiée

Main reste sain.



Moindre privilège

Seulement l'accès dont il a besoin.
Aucun identifiant de production.



Approbations actives

Pour les commandes risquées :
suppression, configuration, réseau.



Relisez chaque diff

Avant chaque commit.



Aucun secret

Gardez-les hors des prompts et des
fichiers lisibles.



Commitez des points de sauvegarde

Revenez rapidement à un bon état.

Mise en pratique : agents sur des tâches complexes

2.0 Mettre en place le projet du Jour 2 — 10 min

2.1 Refactoring à grande échelle avec un agent — 40 min

2.2 Tests avancés : score de mutation, cas limites, propriétés — 30 min

2.3 Mini-atelier d'orchestration multi-agents (stretch) — 25 min

APRÈS CE TP

- Gardez vos mesures de temps : elles alimentent la discussion sur le multi-agent
- Module suivant : intégrer l'IA dans le workflow d'équipe

JOUR 2 • MODULE 6

Intégrer l'IA dans le workflow d'équipe

Conventions, revue de code, documentation, risques et collaboration.



Conventions d'équipe pour l'usage de l'IA



Quels outils sont approuvés ?



Quelles données peuvent, et ne peuvent pas, être partagées avec eux ?



Quand l'usage de l'IA doit-il être mentionné dans une pull request ?



Qui relit, et qui est responsable ?



Où vivent les prompts et les fichiers d'instructions ?



Quelle est la barre de qualité : tests, revue ?



Comment signalez-vous un incident ?



**Une charte d'une page
vaut mieux qu'une
politique de cinquante
pages.**

La discussion compte autant que le document. Relisez-la régulièrement.

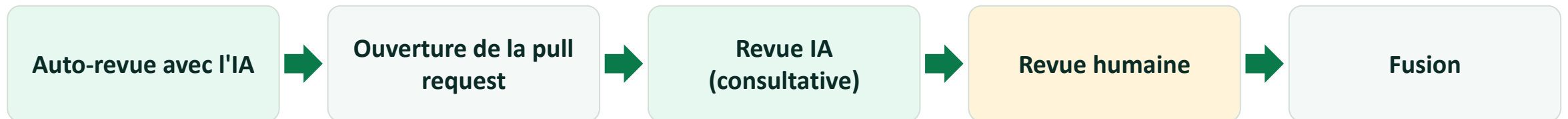
Revue de code assistée par IA

L'IA comme premier relecteur

- Fautes de frappe et bugs évidents
- Vérifications de null manquantes et tests manquants
- Incohérences de style
- Résumés de grosses pull requests

Ce que gardent les humains

- Conception et architecture
- Règles métier
- Jugement de sécurité
- Maintenabilité et intention



La revue IA complète la revue humaine ; elle ne la remplace pas.

Documentation générée automatiquement



README et guides

Installation, usage, onboarding pour les nouveaux collègues.



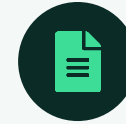
Docstrings et documentation d'API

Générées à partir du vrai code.



Notes d'architecture

Brouillons de comptes-rendus de décisions.



Notes de version

Changelogs à partir des commits et des pull requests.

1

Générer à partir du code

Donnez à l'outil les vrais fichiers.

2

Vérifier l'exactitude

Un document faux est pire que pas de document.

3

Gardez-la dans le dépôt

Elle change avec le code.

4

Mettez-la à jour quand le code change

Faites-en une partie du workflow.

Gestion des risques pour le code généré par IA

Risque	Mitigation
Fuite de données confidentielles	Uniquement des outils approuvés ; règles de données claires ; aucun secret dans les prompts
Vulnérabilités dans le code généré	Revue de sécurité, analyse statique et tests, comme pour du code humain
Licence et propriété intellectuelle	Comprenez les conditions de l'outil et la politique de l'entreprise ; impliquez le service juridique si nécessaire
Dépendances hallucinées	Vérifiez que chaque nouvelle dépendance existe, est maintenue et est la bonne
Sur-dépendance, perte de compétences	Continuez à relire et à comprendre ; continuez à pratiquer les compétences fondamentales
Conformité et auditabilité	Gardez une traçabilité de l'usage de l'IA ; alignez-vous avec les exigences de l'entreprise

Collaboration humain-IA : qui fait quoi ?



Ce que vous apportez

- Intention : quel problème, et pourquoi
- Jugement : ce qui est suffisamment bon
- Responsabilité : votre nom est sur le commit
- Connaissance métier
- Compromis : vitesse, coût, risque, maintenabilité



Ce que l'IA apporte

- Vitesse
- Large connaissance des motifs
- Rédaction infatigable
- Un premier jet sur de nombreuses tâches
- Résumés et traduction entre formats



L'IA rédige. Les humains décident.

Mise en pratique : des conventions d'équipe versionnées

2.4

Fichier d'instructions, modèles de prompts, charte, checklist de revue — 35 min

APRÈS CE TP

- Committez vos actifs d'équipe sur la branche team-assets
- Module suivant : automatiser le cycle de développement avec l'IA

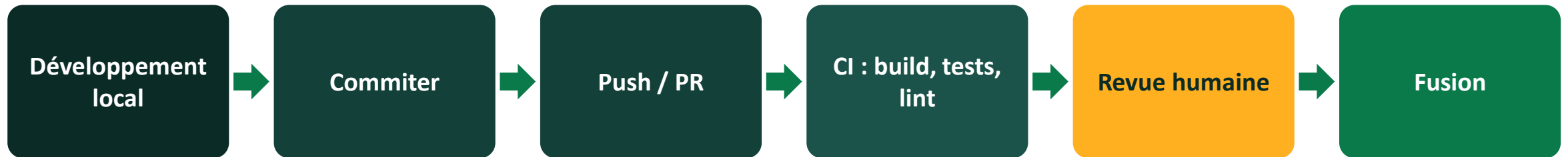
JOUR 2 • MODULE 7

Automatiser le cycle de développement avec l'IA

Scripts, hooks, CI/CD et bots de revue de pull requests.



Où l'IA peut s'intégrer dans votre pipeline



- 1 Scripts de développement**
Messages de commit, résumés de diff
- 2 Git hooks**
Au moment du commit ou du push
- 3 L'IA en CI/CD**
Expliquer les échecs, notes de version
- 4 Bots de revue de PR**
Commentaires de revue consultatifs
- 5 Mises à jour de documentation**
Déclenchées par le changement



Les portes déterministes restent en place : tests, lint, revue humaine. L'IA est un signal supplémentaire, pas un remplacement.

Scripts de développement assistés par IA

- Bons candidats : message de commit à partir du diff indexé, résumé de diff, explication de log CI, notes de version, scaffolding.
- Envoyez le minimum de contexte. Le diff, pas le dépôt.
- Journalisez les entrées et sorties. Pour pouvoir déboguer.
- Gérez les échecs avec élégance. Ne bloquez jamais le développeur silencieusement.
- Suggérez, n'agissez pas. Gardez l'humain dans la boucle.

PSEUDO-CODE

```
#!/usr/bin/env bash
# Pseudo-code : adaptez à votre outil approuvé
DIFF=$(git diff --staged)

MSG=$(echo "$DIFF" | votre-cli-ia \
  --prompt "Rédige un message de commit
  conventionnel pour ce diff. Titre : 72 car.
  max.")

echo "Message de commit suggéré :"
echo "$MSG"
# Le développeur décide s'il l'utilise.
```

Git hooks avec l'IA

Moment du hook	Ce que l'IA peut faire	Précaution
Message de commit	Suggérer un message à partir du diff indexé	Une suggestion ; le développeur édite et décide
Avant un commit ou un push	Revue rapide des changements indexés	Doit être rapide et tolérer l'échec
Avant un commit	Signaler des erreurs probables, comme un secret commité	Préférez des scanners déterministes pour un blocage strict



Rapide

Un hook lent sera contourné.



Tolérant à l'échec

Si l'IA est injoignable, ne bloquez pas.



Consultatif

Les vérifications déterministes bloquent ; les vérifications IA conseillent.



Règles de données

Les mêmes règles s'appliquent aux données envoyées.

L'IA en CI/CD

Bons usages

- Commentaires de revue sur les pull requests
- Suggestions de tests manquants
- Triage des échecs en langage clair
- Changelogs et notes de version
- Vérifications que la documentation correspond au changement

1

Consultatif avant bloquant

2

Les vérifications déterministes restent la porte de contrôle

3

Secrets dans le coffre CI ; permissions minimales

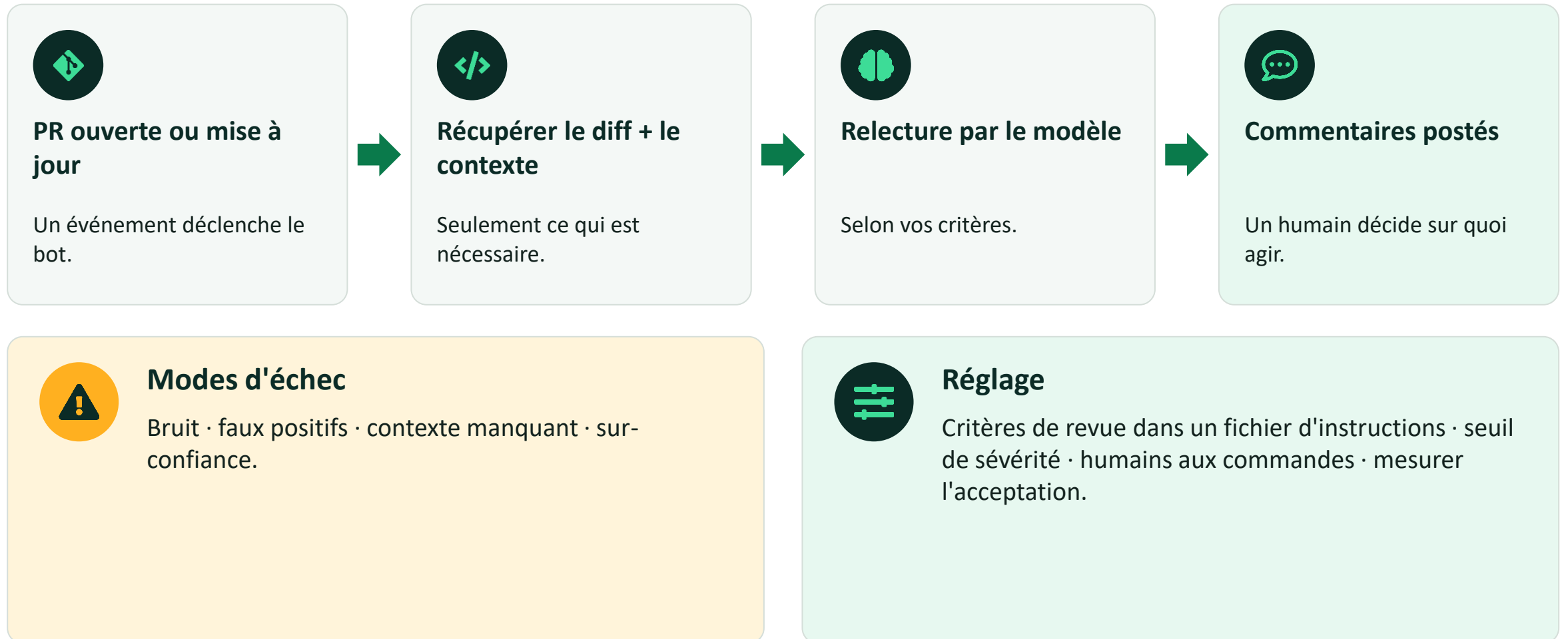
4

Traitez le texte des pull requests comme non fiable

5

Envoyez seulement ce qui est nécessaire ; surveillez le coût

Bots de revue de pull requests



Mise en pratique : automatiser le cycle de développement

2.5

Hook, brouillon de CI, bot de revue — 35 min

APRÈS CE TP

- Gardez vos six scénarios de test du hook pour le debrief
- Module suivant : choisir sa stack IA de développement

JOUR 2 • MODULE 8

Choisir sa stack IA de développement

Agents vs IDE assistés par IA, critères de décision et standardisation.



Agents vs IDE assistés par IA, comparés

Dimension	IDE assisté par IA	Agent autonome
Idéal pour	Travail dans le flux ; éditions visibles et immédiates	Tâches multi-étapes et multi-fichiers
Contrôle	Vous approuvez chaque changement au fur et à mesure	Vous relisez un résultat après plusieurs étapes (configurable)
Contexte	Fichiers ouverts et espace de travail	Explore le dépôt par lui-même
Autonomie	Plus faible	Plus élevée
Courbe d'apprentissage	Familier : vit dans votre éditeur	Nouvelle habitude : donner un objectif, relire un résultat
Profil de risque	Surtout de petits changements visibles	Changements plus gros : les garde-fous comptent davantage
Adéquation avec l'équipe	Flux quotidien	Tâches plus grosses. Beaucoup d'équipes utilisent les deux

Des tendances typiques, pas des vérités absolues. Les deux familles convergent.

Choisir selon le type de projet, la taille d'équipe et la sécurité

Situation	Orientation
Grosse codebase legacy	Agent pour l'exploration ; assistant IDE pour les éditions
Petit prototype greenfield	Les deux conviennent ; un agent peut être rapide pour le scaffolding
Petite équipe	Léger et flexible, avec des conventions partagées
Équipe plus grande, ou plusieurs équipes	Standardisez-vous sur un ou deux outils approuvés, configuration centrale
Confidentialité stricte	Uniquement des options approuvées ; vérifiez d'abord les flux de données (options privées ou locales, selon la politique)
Logiciel réglementé ou critique	Des portes de revue plus fortes et une piste d'audit claire



Quelle que soit la situation : lancez d'abord un petit pilote, sur une tâche réelle mais à faible risque, et décidez sur la base de preuves.

Standardiser les outils dans l'équipe



Standardiser

- La liste des outils approuvés
- Les fichiers d'instructions partagés dans le dépôt
- Une bibliothèque de prompts partagée
- Une checklist de revue partagée
- Un guide d'onboarding
- Une revue régulière de vos choix

Laisser libre

- Préférences personnelles d'éditeur
- Thèmes et raccourcis

Standardisez les entrées et les sorties,
pas le workflow personnel.

Mise en pratique : choisir et standardiser sa stack IA

2.6

Noter les 3 approches, appliquer vos contraintes, définir un pilote
— 15 min

APRÈS CE TP

- Gardez votre tableau de notation et votre plan pilote
- Pour finir : le projet fil rouge complet

Synthèse du Jour 2 : assemblez votre workflow IA complet



✓ Tests automatisés générés avec l'IA



✓ Projet structuré : README, documentation, conventions



✓ Workflow de développement automatisé



✓ Revue de code assistée par IA en place



→ TP 2.7 : construisez et présentez votre fiche de workflow IA complet

VOS LIVRABLES DU JOUR

- Un fichier d'instructions pour le projet
- 2 à 3 modèles de prompts
- Une checklist de revue
- Un brouillon d'intégration CI ou hook

Habitudes de sécurité : branche, moindre privilège, relire chaque diff, aucun secret.

CONCLUSION

Dix règles à retenir

1 Un LLM prédit des tokens. Plausible n'est pas vrai.

2 Le contexte, c'est tout. Pertinent bat grand.

3 Agent = modèle + outils + boucle. Le harnais agit.

4 Un prompt est une spécification : six blocs.

5 Petites étapes, tests et relecture du diff.

6 Adaptez l'outil à la tâche et aux besoins de sécurité.

7 Vous êtes propriétaire de chaque ligne que vous livrez.

8 Les connaissances de l'équipe vivent dans le dépôt.

9 Automatisez progressivement : consultatif d'abord.

10 Réévaluez régulièrement : ce domaine évolue vite.

Votre plan d'action



Cette semaine : pour vous-même

Utilisez le prompt en six blocs sur une tâche réelle.

Écrivez un fichier d'instructions pour votre dépôt principal.

Appliquez les cinq vérifications avant d'accepter du code.



Prochaines semaines : avec votre équipe

Rédigez une charte IA d'une page.

Partagez 2 à 3 modèles de prompts dans le dépôt.

Mettez-vous d'accord sur les outils approuvés et les règles de données.



Plus tard : passer à l'échelle avec prudence

Essayez une revue IA consultative sur les pull requests ou en CI, et mesurez-la.

Planifiez des revues régulières de vos outils et pratiques.



Nommez une action que vous allez entreprendre la semaine prochaine. Écrivez-la dans le chat.

Évaluation, certificat et suivi



Évaluation

Basée sur le travail pratique que vous avez partagé avec moi en temps réel.



Bilan de formation

Envoyé par email à chaque participant, avec l'évaluation du formateur.



Certificat de réussite

Envoyé par email, à partager si vous le souhaitez.

Suivi optionnel

- Rendez-vous téléphoniques réguliers pour répondre à vos questions
- Présence sur site au démarrage de votre projet
- Un audit de vos pratiques quelques semaines ou mois plus tard

MERCI, CGG SERVICES SAS

Questions ?

Comprenez-le. Donnez-lui du contexte. Vérifiez-le.
Assumez-le.



IA pour les devs · Maîtrisez l'IA pour coder